Creation of an AI Maze Solver

Abstract

This artefact explores the development of an artificial intelligence agent capable of solving a randomly generated maze by using reinforcement learning algorithms. The project can be split into 3 major sections, the first being the production of a robust maze-generation system using core principles in graph theory. The second part involves the writer choosing and creating an adequate maze solving algorithm followed by the writer actually creating the AI maze solver. In the final section the writer ties all these concepts together and creates a user interface, to better help interact with and demonstrate some of the crucial concepts.

Introduction

In this modern day and age, AI (artificial intelligence) is one of the most frequently discussed topics. Furthermore, the need to automate traversal across unknown regions grows larger and larger every day (due to the growing population and greater need to transport goods). To combat this, the technology industry has begun pushing youth into software development and robotics to incentivise innovative thinking around solving this problem (among many other problems.) My younger brother Aleks, will be participating in a maze solving competition where he needs to design and code a robot with the ability to solve any given maze. Consequently, he has come to me with the task of building an AI to solve a maze. The artefact is a proof-of-concept program which is designed to provide him with a maze learning solution. This addresses his need to solve the maze without having a top-down perspective of the maze. This issue does arouse some large controversies. For instance, many people fear that AI may cause them to lose their job (AI replacing truck drivers.) Moreover, people also fear the potential dangers of fully trusting an AI with heavy machinery given that tech is known to malfunction (could cause injury). However, I am not making any industry grade AI that could injure or replace workers and will be more focused on theoretically solving a maze using an AI for my extended project qualification (EPQ.) Currently, I am studying maths, further maths, physics and computer science A levels so this EPQ complements my A Levels. I also currently aspire to work in the tech industry (as a programmer of some form) and so I believe this EPQ will also help me broaden my understanding of this industry. This task, though easy to say, will not be easy to execute and thus I believe it is crucial that I lay out some key objectives in the development process:

1) I must first install and experiment with the libraries that I will be using for my project.
2) I then need to generate the maze using standard maze generations techniques.
3) Next, I will have to solve the maze using pathfinding algorithms to provide a benchmark, to compare my agents to.
4) I will then need to train and test an AI using the maze in order to solve the maze.
5) Finally, I will make a visual UI to make the whole program a lot more understandable.

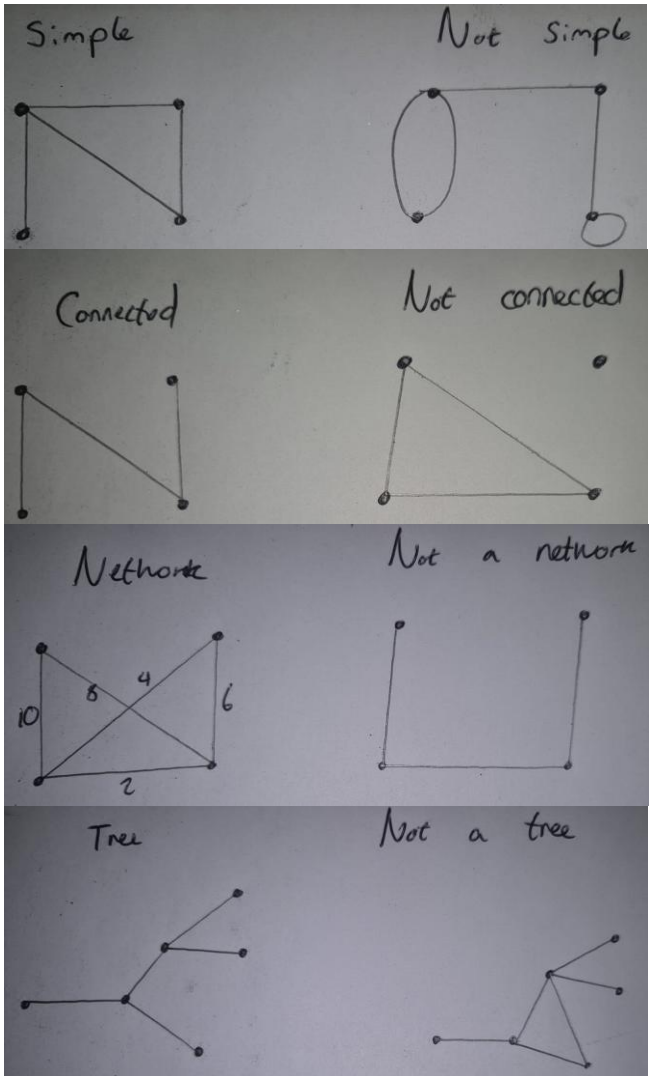
Research Review

To begin this daunting task, I must ensure that my research is in depth and clinical. Questions such as: ‘what is a maze?’, ‘How do I represent a maze?’ or 'how do I even begin

to automate solving a maze?' immediately begin to arise. In computer science, decomposition is a well-known problem solving methodology. Decomposition is the process of breaking down a large problem into smaller parts. So instead of asking these very broad questions, I will begin by focusing on how to even represent a maze.

Graph Theory

Graph Theory is one branch of mathematics that involves studying the complex relationship between objects in a graph [13]. This graph is not your standard cartesian (x,y) graph, but is instead a discrete set of points known as nodes (or vertices), joined together by lines which are known as arcs (or edges) [13]. A loop occurs when a node has an arc connecting to itself [13][14]. Furthermore, if two nodes are connected by more than one arc, the pair is known to have 'multiple edges' [13]. A simple graph has no loops and no double edges [13][14]. A connected graph is a graph, where you can get to any node starting from any node by travelling across some combination of arcs [13][14]. Upon observing the structure of a graph, one can notice its striking similarities to a maze. Each arc in the graph could be used to represent a corridor in a maze and each node could be the beginning or end of a corridor. A maze would have to be a simply connected graph, because mazes do not contain looping or double edged corridors. Furthermore, to allow the person to escape the maze, it must be possible to start from any point and arrive at any other point, implying that the graph must be connected. There is however, one major issue that currently exists with the current representation of a maze. In real life, mazes have corridors of varying lengths, whereas in my current representation, each 'corridor' has the same length. To overcome this, we will use a different type of graph known as a network. A network is simply a weighted graph, meaning that each arc is given a numerical value [14]. In my case, my maze would be a network, where the weight of each arc represents the length of each corridor. Both a network and a graph can be represented as an adjacency matrix instead of a visual graph to make it easier for a computer to understand [13]. The final piece of essential graph theory for this project is the concept of a tree, this being a graph that has no loops or cycles [13][14]. Later on, when partially generating the maze using Prim's algorithm, I will have produced a minimum spanning tree, which will be key when producing a realistic maze [3][4].



Breadth-First Traversal (Maze generation part 1)

As stated in the previous paragraph, in order to produce a solvable maze, the graph that I need to generate needs to be connected. To achieve this, I will use a breadth-first traversal. Before this algorithm can be explained, I must explain how data is stored and manipulated in standard programming languages. In computer science, there are 3 data types [17]. Elementary data types are basic, for instance an integer is an elementary data type [17]. Composite data types are made from combining multiple elementary data types, for instance, an array of multiple integers stored side by side [17]. The final data type is an abstract data type. An abstract data type stores data, but more importantly, is a description of how we access and use this data [17]. A queue is one of these abstract data types and operates similarly to how a queue works in real life. Items can only be added to the back of the queue (greatest index) and can only be accessed from the front of the queue [17][18]. In a breadth-first traversal, there is one queue that will store nodes to be visited, and there is one list which stores nodes that have already been visited [1][2]. The first node is added to the list and all the first node's neighbouring nodes are added to the queue [1][2]. The node from the front of the queue is removed from the queue and added to the list (indicating that we have traversed to this node) [1][2]. All the neighbouring nodes of this new node (that aren't already in the queue or the list) are added to the back of the queue [1][2]. This process is repeated until there are no more nodes left in the queue, which indicates that the graph has been fully traversed [1][2]. So to begin generating my maze, first, I will generate a completely random adjacency matrix. I will then perform a breadth-first traversal through this adjacency matrix. If the number of nodes in my breadth-first list is equal to the number of nodes that have been randomly generated, then the random graph that has been generated will be connected (the first stage has been successful). If not, then a new random adjacency matrix will be generated.

One of the other algorithms that I could have chosen in this project instead of the Breadth-First Traversal algorithm was the Depth-First Traversal algorithm. This algorithm works by going as 'deep' into the graph as possible and then going back on itself to explore any remaining unexplored nodes [1][2]. Both algorithms have the same time complexity of O(V+E) [2] so this was not a factor that I considered when choosing between the two. However, the Breadth-First algorithm has a better auxiliary space complexity of O(V) [2] in comparison to the O(V+E) [2] of the Depth-First algorithm. The DFS algorithm also comes with the additional problem that it would need to be written recursively [2]. This not only makes the DFS significantly harder to code, but it also comes with the potential risk of a stack overflow error if the generated mazes were to become too large.

Prim's Algorithm (Maze generation part 2)

At this point, I have a randomly generated 'maze' that is connected meaning it can be solved. Saying this, the generated graph will have way too many arcs to the point where it will not look like a reasonable maze. The number of these arcs needs to be reduced and this will be done using Prim's algorithm [3][4]. In Prim's algorithm, you start from any vertex, and you select the shortest arc connecting that vertex to another neighbouring node [3][4]. Between the visited nodes, you choose the next shortest arc connecting one of the nodes to a new node so that there are no loops or cycles [3][4]. This process is repeated until the

graph is connected - which can be determined because the number of arcs will be one less than the number of nodes [13]. At this point I have a minimum spanning tree which represents the maze. However, the maze is ridiculously easy to solve because there are no loops or cycles . Therefore, in my program, I will introduce a small probability (~10% or 20%) that a node removed from the original connected graph is reintroduced back into the new minimum spanning tree. This will stop the graph from being a tree anymore (as now the graph will have loops), and will now make the graph begin to look like a realistic maze.

Similar to previously, I also had the choice between Prim's algorithm and Kruskal's algorithm when generating the minimum spanning tree from the connected graph [3]. Kruskal's algorithm works by re-adding arcs back into the graph in increasing order of weight so that no loops or cycles are formed [3]. Both algorithms have the same time complexity of O(n^2) [3] so this was not a factor considered when actually choosing between the two. Kruskal's algorithm, although simpler to understand, would require me to constantly loop through the graph after every addition of an arc to verify that the graph is still a tree and does not have a loop [3]. This can be done efficiently in near constant time complexity however, would still be difficult to do. Prim's algorithm does not have this issue, which made it an easier algorithm to implement and hence I chose this algorithm to actually use.

Dijkstra's Algorithm (Maze solving)

As a result of the previous two algorithms, we now have a network that can represent a realistic, solvable maze. When training the reinforcement learning model, I will need graph solving benchmarks to compare the agents against, hence I need the time to solve the maze that is considered 'good' to reward effective agents [7][8][19][20]. Thus I need to actually solve the maze mathematically to find the shortest path before I train the agents and in order to do this, I will use Dijkstra's algorithm [5][6]. How Dijkstras works is: each node is given 3 values: the order in which that particular node was turned into a fixed node; the distance to that node once it became a fixed node, otherwise known as the permanent label; and the current shortest distance to that node known as the temporary label [5]. The starting vertex immediately becomes a fixed node [5][6]. It has an order of 1 and a permanent label of 0. The distance between this fixed vertex and each neighbouring node is added to the temporary label of the respective neighbouring vertex [5][6]. The vertex with the smallest value of the temporary label is turned into the next fixed node [5][6]. It has an order of 2 and its permanent label takes the value of the temporary label [5][6]. Now, all the neighbours of this new fixed vertex are observed [5][6]. If a neighbour is already a fixed vertex, it is completely ignored [5][6]. If the neighbouring node is not a fixed node, a new distance is calculated [5][6]. This distance is made by adding the distance between the two vertices to the value of the permanent label of the new fixed vertex [5][6]. If the neighbour has no values in the temporary label, the temporary label is set to this new distance [5][6]. If the neighbour has a value in the temporary label, if the new distance is smaller than this temporary label, then the temporary label is set to the new distance, otherwise the new distance is ignored [5][6]. After all neighbours of this fixed node have been observed, a new fixed vertex is chosen and the process is repeated [5][6]. This repeats until the end vertex is turned into a fixed vertex [5][6]. At this point, the value in the permanent label of the end vertex is the distance of the shortest path between the starting vertex and the ending vertex. For my program, this shortest path length value would be sufficient to train the model against. For

completeness though, the algorithm should also perform a backwards pass. Starting from the end vertex, you observe all neighbouring nodes of the end vertex. If the (permanent label of the end vertex)-(permanent label of neighbouring vertex)=(arc length between the two nodes), then we know that that neighbouring vertex was part of the shortest path [5][6]. This process is repeated, now using that neighbouring vertex and using all of its neighbours [5][6]. This continues, until we have returned to the starting vertex, giving us the nodes we need to traverse for the shortest path from the starting vertex to the end vertex. This algorithm always finds the shortest path between two nodes [5][6]. The A* algorithm that is used to find the shortest path between points in applications such as google maps is based entirely on Dijkstra's algorithm [21][22]. The reason I am not using the A* algorithm over Dijkstra's algorithm even though the A* algorithm is more efficient is because the A* algorithm relies on actually calculating the geometrical distance between the neighbouring node and the end node as well as using the distance between two vertices to be traversed [21][22]. However, because my graph will not operate using x,y coordinates, there will be no way for me to calculate this geometric distance hence making A* is not possible.

Reinforcement learning AI

In recent years, AI has been a term that has been heavily overused in industry. AI itself stands for artificial intelligence and is a program that is designed to perform tasks that would generally require human intelligence. This is an umbrella term and there are actually many different types of AI. For instance, a fundamental form of AI in statistics is a linear regression model. You have a set of input and output values (training data) and a linear regression line (line of best fit) is drawn to best fit these values. An AI model can now interpolate from this regression line using a new input to predict the value of an output. The AI is not ‘thinking’ as a human would but is instead using training data to make predictions about new data.

For my project though, I will need to use a slightly more complicated form of AI known as reinforcement learning. To aid with the rough concept, you can imagine this model as an animal that is designated to perform tasks. If the animal performs the task well it receives a reward, otherwise it receives a punishment. Based entirely off of these rewards and punishments, the animal can make better decisions in the future (and perform the task more effectively). In reinforcement learning, there is an agent and an environment, the agent being the model making the decisions based on the environment [7][8]. The cycle of reinforcement learning is, the environment provides the agent with a state, the agent responds to the state with a corresponding action, then the environment responds to this action with an appropriate reward or punishment [7][8]. But one may ask, how can I programmatically make an AI make a choice between action? This is where the Q-Function comes in. The Q-Function takes in 2 parameters, the current state and a potential action [7][8]. The Q-Function then calculates a score based on these 2 parameters. This score represents the total reward from this moment onward to the final state of the environment if the agent were to perform this action [8]. This currently seems rather nonsensical because the total reward from this action is also dependent on future actions after this action [8]. To solve this issue, we split this reward into 2 parts. The agent receives an immediate reward for performing an action and the agent is also assigned an appropriate policy alongside the q-function [7][8]. A policy is a function ($\pi(s)$ notation) which simply tells the agent what to do from this moment onwards [8]. An example of a fixed policy when solving a maze could be, always turn right from now (fixed policies are rarely used though and we will use state-dependent policies) [8].

The standard Q-Function policy for reinforcement learning ai is to choose the action in the following state with the highest Q-Function value. [7][8] The agent then chooses the action that has the highest Q-function score. All this above can be summarised by the Bellman equation as shown below:

$$Q(s,a) \leftarrow Q(s,a) + \alpha(r + \gamma \max Q(s',a') - Q(s,a))$$

The alpha in this equation outside of the brackets is the learning rate of the agent [7][8]. To simplify the explanation, we will assume that the learning rate of the AI is 1, which significantly reduces the complexity of the equation to:

$$Q(s,a) = r + \gamma max_{a'} Q(s',a')$$

In this equation, **s** stands for the current state of the agent, **a** stands for the potential action that the agent may take [7][8]. **s'** and **a'** stand for the respective state and action in the next move, given that this first action occurs [7][8]. **r** stands for the immediate reward of performing the first action and **γ** stands for the discount factor [7][8]. As mentioned above, what this equation means is, the value of the Q-Function is equal to the immediate reward of performing the action plus the maximum discounted value of the q function of the next state. The reason there is a discount factor is because immediate rewards are considered 'more valuable' than rewards in the far future [8]. However, we cannot directly use the Bellman equation because it is defined recursively and relies on knowledge of all future state-action pair sequences, which is not feasible in most environments because there is usually an infinite number of these successions. This is where we introduce the Tabular Q-Function. Rather than trying to calculate the value of the Q-function using recursion and the Bellman equation, we will approximate the value instead [7][8]. In the table, we store every single possible state action pair and we assign it a Q-Function value - which we believe is the best approximation for the true value [7][8]. Now, whenever the function is called, this approximation is used instead. Initially, when we set off our agent through the maze, it will have a random set of Q-function approximations which will mean it will be awful at solving the maze but, each time the agent attempts to solve the maze, it will adjust some of these approximations so that they fit the bellman equation more appropriately, hence training the agent to solve the maze [7][8]. After many iterations of the agent, we should have an AI which is capable of solving the maze. This is a slight oversimplification of the training of the agent and has overlooked the epsilon greedy policy. When the q-function table is created, all the q-values will initially be 0, so the agent won't discover better actions beyond choosing the first available action. This makes sense because the AI cannot pass through the maze using paths it remembers from the maze if it has never passed through the maze. Therefore, each action the agent performs can be split into two categories: exploration and exploitation. In an exploration step, the agent takes a step in a completely random direction irrespective of the q-values in the table. In an exploitation step, the agent uses the table (its 'memory') to decide which node to move to. Over the duration of the program, the value of epsilon changes to ensure that the agent slowly begins to prioritize exploitation over exploration.

If you read further into reinforcement learning, you will find the tabular q-function is not the only type of RL model but there is also Deep-q RL [19][20]. This model is significantly more complex than the model I am using and involves using neural networks rather than tables [19][20]. The major issue with the tabular model is, as the maze grows larger, the size of the table which stores the state action pairs grows exponentially, making the table infeasible for

these larger mazes [19]. Using neural networks allows deep learning models to overcome this issue, and means the models can solve these larger mazes [19]. However, the process of making a neural network is an incredibly difficult task which requires a high level knowledge of math and linear algebra. Furthermore, the mazes in my brother's robotics competition will not grow to the scales I am describing therefore, I believe it was unnecessary to make a deep-q-learning model.

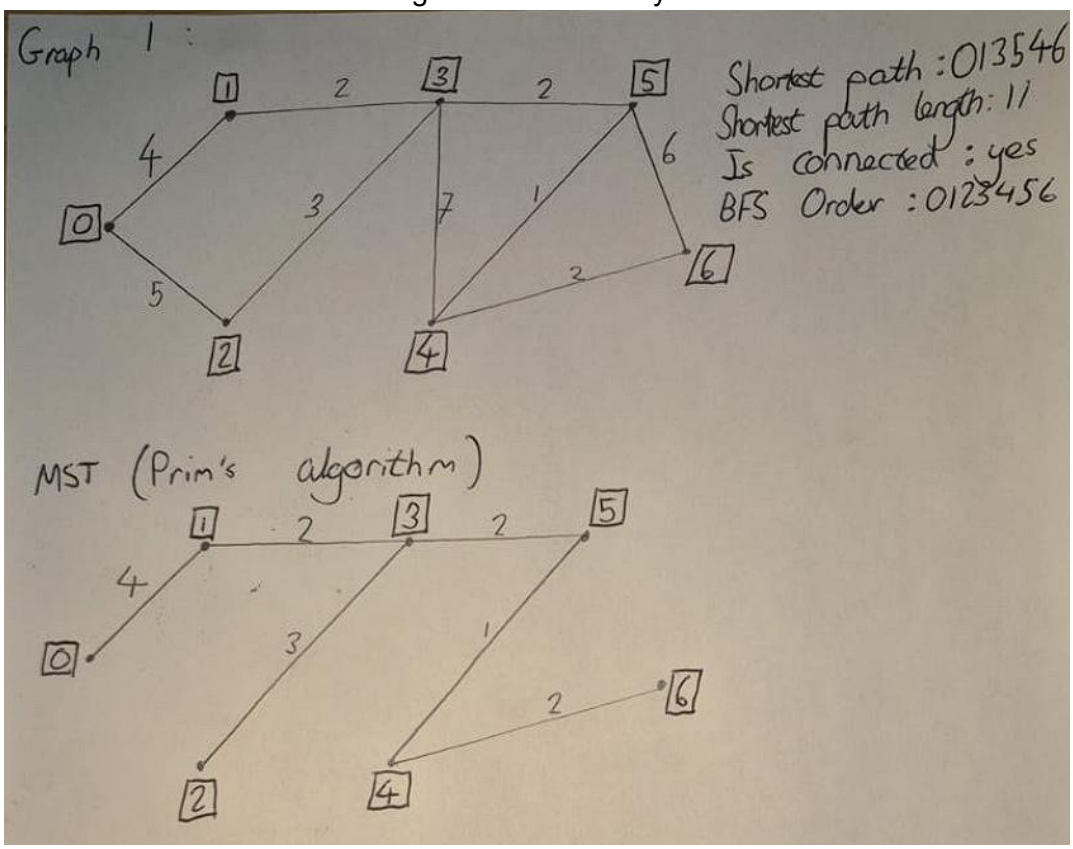
Development

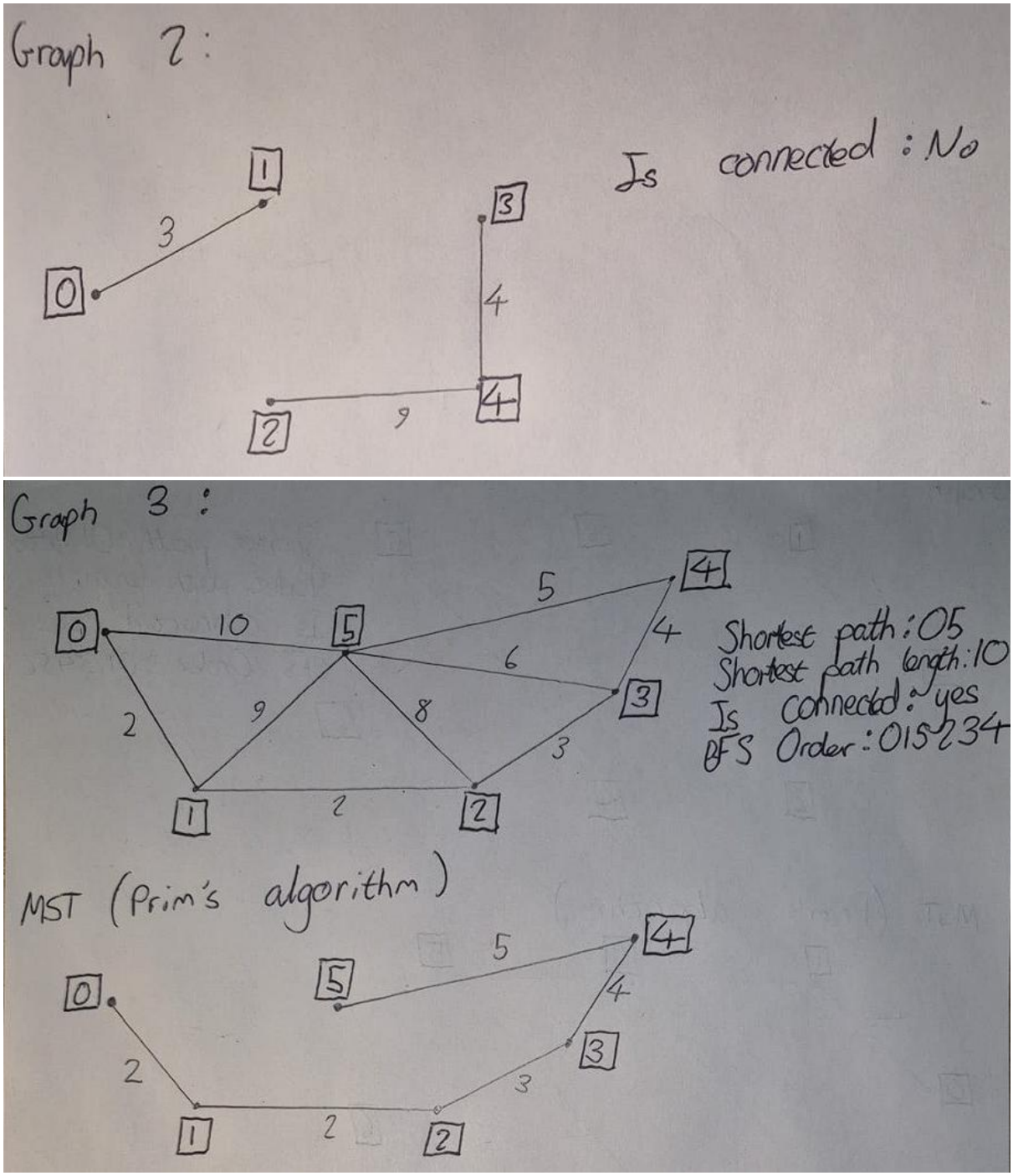
At this stage in the EPQ most of my burning questions had been answered and I was ready to begin the programming with my newfound knowledge. I split my programming into 3 major sections:

- Maze generation/Graph algorithms
- AI maze solving capabilities
- User interface

Maze generation and graph algorithms

To make testing easier, I decided to create most of the graph algorithms first before concerning myself with the random nature of the maze. Therefore I had to create 3 graphs that could be used to test the algorithms. Here they are:





Graphs 1 and 3 were used to test whether prims or dijkstra's algorithm worked effectively whereas graph 2 was used to test the breadth first search. I decided to represent each graph using 2D arrays acting as adjacency matrices. Here they are:

Graph 1:
[
[-1,4,5,-1,-1,-1,-1],
[4,-1,-1,2,-1,-1,-1],
[5,-1,-1,3,-1,-1,-1],
[-1,2,3,-1,7,2,-1],
[-1,-1,-1,7,-1,1,2],

[-1,-1,-1,2,1,-1,6],
[-1,-1,-1,-1,2,6,-1]
]

Graph 2:
[
[-1,3,-1,-1,-1],
[3,-1,-1,-1,-1],
[-1,-1,-1,-1,9],
[-1,-1,-1,-1,4],
[-1,-1,9, 4,-1]
]

Graph 3:
[
[-1,2,-1,-1,-1,10],
[2,-1,2,-1,-1, 9],
[-1,2,-1,3,-1,8],
[-1,-1,3,-1,4,6],
[-1,-1,-1,4,-1,5],
[10,9,8,6,5,-1]
]

After I made each matrix I could begin creating a breath-first-search algorithm to verify that the maze is connected.

Here is the algorithm:

```
1   def isConnectedBFS(self, graph):
2       #This function performs the breadth first search (for the randomly generated graph) tpo determine if the graph is connected.
3       #These nested for loops convert the adjacency matrix into an adjacency list.
4       adjacencyList = {}
5       count = 0
6       for row in graph:
7           tempList = []
8           for index, item in enumerate(row):
9               if (item != -1):
10                  tempList.append(index)
11          adjacencyList[count] = tempList
12          count += 1
13      #The queue stores the nodes in the graph that are yet to be visited. The visited list stores the nodes that have already been visited.
14      queue = [0]
15      visited = []
16      #As long as the queue is not empty, the first node in front of the queue is removed. This node is added to the back of visited and its
neighbours are added to the back of the queue. The neighbours are only added if they have not already been visited and are not already in the
queue.
17      while (len(queue) > 0):
18          currentIndex = queue.pop(0)
19          visited.append(currentIndex)
20          nN=adjacencyList[currentIndex]
21          for n in nN:
22              if (n not in visited and n not in queue):
23                  queue.append(n)
24      if (len(visited) == len(graph)):
25          return True
26      else:
```

```
27          return False
```

This code snippet was removed directly from the class which is why there is a self in the parameters however this doesn't affect the functionality of the algorithm. To begin the algorithm I decided to convert the adjacency matrix into an adjacency list. An example of an adjacency list for graph A is shown below (stored as a map):

```
{
        0: [1,2],
        1: [0,3],
        2: [0,3],
        3: [1,2,4,5],
        4: [3,5,6],
        5: [3,4,6],
        6: [4,5]
}
```

The loop used to execute this is found in lines 4-12 and basically, all I do is for each row in the matrix, if a connection found at column n is not equal to -1, then I add column n to the list of the correct row in the map. At this point I define a queue (FIFO Abstract Data Type) which will contain all the nodes to be visited. I also defined a new list containing all the nodes visited during the traversal. By default the queue had node 0 inside it so that the algorithm would have a feasible starting point. Then, as long as the queue is empty the front item is removed from and all the neighbours of the new visited node that have already not been visited or are not yet to be visited are added to the queue. This occurs in the while loop between lines 17-23. Finally, if the length of the visited nodes list is equal to the number of items there are in the graph, then all the items in the graph have been traversed to, therefore the graph is complete and the function returns a value of True. Upon implementing and testing the algorithm, it returned true for graph 1 and graph 3 and returned false for graph 2, indicating that the algorithm worked correctly.

At this point, I was able to filter between connected and unconnected graphs meaning that any maze I was to generate would be solvable. At this point I needed to convert any connected graph into a minimum spanning tree, to make it into a more realistic maze and this was done using Prim's algorithm:

```
1   def primsAlgorithm(self,graph):
2       addedArcs = {}
3       availableColumns = [0]
4       count = 1
5       #The while loop runs until all the nodes have been added to the minimum spanning tree. For each loop, the smallest arc from the available
columns is found and added to the addedArcs dictionary.
6       while (len(availableColumns) != len(graph)):
7           smallestArc = CONST_BIGNUMBER
8           rowOfSmallest = -1
9           columnOfSmallest = -1
10            for i, row in enumerate(graph):
11                if i not in availableColumns:
12                    for j, column in enumerate(row):
13                        if j in availableColumns:
14                            if graph[i][j] != -1 and graph[i][j] < smallestArc:
15                                smallestArc = graph[i][j]
16                                rowOfSmallest = i
17                                columnOfSmallest = j
18          availableColumns.append(rowOfSmallest)
19          newInfo = [columnOfSmallest,rowOfSmallest, smallestArc]
20          addedArcs[count] = newInfo
```

```
21          count += 1
22      #An empty matrix is created with the dimensions of the original graph.
23      mst = []
24      for i in range(len(graph)):
25          row = []
26          for j in range(len(graph)):
27              row.append(-1)
28          mst.append(row)
29      #The contents of the addedArcs dictionary are inserted into the empty matrix to create adjacency matrix for the minimum spanning tree.
30      for index in addedArcs:
31          mst[addedArcs[index][0]][addedArcs[index][1]] = addedArcs[index][2]
32          mst[addedArcs[index][1]][addedArcs[index][0]] = addedArcs[index][2]
33      return [mst, addedArcs]
```

Although it looks complicated, this algorithm is actually pretty straightforward to follow. Initially I set up a few necessary variables and data structures such as a map to store edges in the minimum spanning tree and the nodes that are available in the current iteration of the minimum spanning tree. Then, as long as the number of nodes in the current minimum spanning tree doesn't match the number of nodes in the graph, I will do 1 iteration (lines 6-21) of Prim's algorithm to add a new node to the minimum spanning tree. In this iteration, the algorithm passes through all possible neighbours outside of the MST for each node in the current MST and selects the neighbour which requires the shortest path length to get to (lines 8-17). I now add this node to the list of nodes in the current minimum spanning tree. I also added the information for the new connection to my map (lines 18-21). At this point I have a map that contains the info of all the arcs in the minimum spanning tree. The algorithm needs to convert this information back into a new adjacency matrix. It creates a new 2d list with the same dimensions as the initial adjacency matrix, filled with -1's to represent all the nodes in the minimum spanning tree unconnected (lines 24-28). At this point, I pass through all the edges in my addedArcs map and populate the adjacency matrix with appropriate weights according to this map. I tested my prims algorithm on graph 1 and graph 3 and both returned the correct matrices for the minimum spanning tree indicating that the algorithm works correctly.

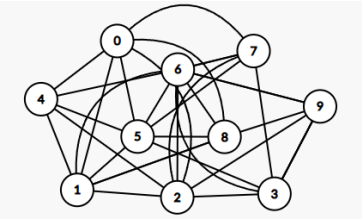
At this point I believed I had the crucial algorithms for maze solving completed and further believed it would be suitable to actually generate a random maze before I began to solve it using either Dijkstra's or using AI. I also began to start placing my functions into an appropriate Maze class. As stated in my research review my maze generation plan was to: generate a random adjacency matrix; verify it was connected properly; use prims algorithm to cut down most arcs in this maze and then re-add only a few of these removed arcs.

Here is the algorithm to generate a random adjacency matrix:

```
1 def generateRandomGraph(self):
2     #This function generates a random graph of size between 10 and 20 nodes. The weight of each arc is between 1 and 10 and there is a 50% chance that the arc is not present (so is -1.)
3     graph = []
4     numberOfNodes = random.randint(10,20)
5     for i in range(numberOfNodes):
6         row = []
7         for j in range(numberOfNodes):
8             row.append(-1)
9         graph.append(row)
10    for i in range(numberOfNodes):
11        for j in range(numberOfNodes):
12            if i != j:
13                if graph[i][j] == -1 and graph[j][i] == -1:
14                    isNegative = random.randint(0,1)
15                    if isNegative==1:
```

```
16                  insert = random.randint(1,10)
17                  graph[i][j] = insert
18                  graph[j][i] = insert
```

The first lines (3-9) create a new adjacency matrix of a random size (10-20) and fill it with -1s. By modifying the number of nodes input parameters, the algorithm will generate a larger random maze, however the visualisation will be less optimal.This represents the generation of a random number of disconnected nodes. Then in the second for loop, the algorithm passes through every possible connection and checks if that connection is empty. If that connection is empty, there is a 50% chance that the connection will be created with an arc length of between 1-10. Currently any graph generated would look something like this:



This graph is clearly no good to represent a maze as there are too many arcs for the few nodes. This is where the code would apply the prims algorithm to convert this graph into a much smaller MST (using the algorithm above.) Prim's algorithm also returns a list of connections between arcs which will be important for the next stage.

At this point I want to re-add a few nodes. This is done in two stages:

- Identifying the unused arcs.
- Re-adding the unused nodes back to the MST.

Here is the code for identifying the unused nodes:

```
1 def identifyUnusedArcs(self, graph, addedArcs):
2    #A new list containing the unused arcs is created. The arcs are only added to the list if they have not been used in the minimum spanning tree.
3    #Ignore I's and ignore J's is used to prevent the same arc being re-added twice.
4    unusedArcs = []
5    ignoreIs = []
6    ignoreJs = []
7    #The function loops through every item in the matrix (excluding items that have already been visited in the other half of the symmetrical matrix). If the item isn't in the addedArcs dictionary, it is added to the unusedArcs list.
8    for i,row in enumerate(graph):
9        for j, column in enumerate(row):
10           if (graph[i][j] != -1):
11               if (i not in ignoreIs and j not in ignoreJs):
12                   isInAddedArcs = False
13                   for index in addedArcs:
14                       if addedArcs[index][0] == i and addedArcs[index][1] == j:
15                           isInAddedArcs = True
16                       elif addedArcs[index][0] == j and addedArcs[index][1] == i:
17                           isInAddedArcs = True
18                   if isInAddedArcs == False:
19                       unusedArcs.append([i,j,graph[i][j]])
20                       ignoreIs.append(j)
21                       ignoreJs.append(i)
22   return unusedArcs
```

The arcs added alongside the adjacency matrix for the original graph are passed into the function. The function creates 3 new lists and then loops through each entry in the original

graph. If this entry is not equal to -1 or exists in the added arcs list, then this arc is not added into the UnusedArcs list. Otherwise, the information for the connection (starting node, ending node and arc length) are added as a list into Unused arcs. If an arc is added into unused arcs, the nodes of the starting and ending index are added into ignore i's and ignore j's. In later passes, these indexes are ignored even if they are valid. This is done to prevent double entries into UnusedArcs due to the graph being symmetrical down the leading diagonal.

With the list containing the unusedArcs, we can now proceed to re-add some of these arcs back into the MST. This is done using this algorithm:

```
1  def reAddArcs(self, mst, unusedArcs):
2      #The function loops through the unusedArcs list. If the random number generated between 0 and 1000 is less than 125, the arc is re-added to the minimum spanning tree.
3      for arc in unusedArcs:
4          if (random.randint(0,1000) <125):
5              mst[arc[0]][arc[1]] = arc[2]
6              mst[arc[1]][arc[0]] = arc[2]
```

To put it simply, this function loops through every arc that was not used in the minimum spanning tree and gives it a 12.5% chance of it being re-added. Initially I had a 10% chance of it being re-added, but after some testing I found that 12.5% was more optimal for creating a more complex maze.

These procedures are stitched together in one larger procedure:

```
1#This function is called when an instance of the class Maze is created.
2      self.maze = []
3      self.shortestPath = ""
4      self.shortestPathLength = 0
5
6      #To begin maze generation, a random graph is created. If the maze is not connected, validated using my personalised breadth first search algorithm, then a new random graph is created.
7      g = self.generateRandomGraph()
8      while self.isConnectedBFS(g) == False :
9          g = self.generateRandomGraph()
10     #The convert to maze function uses prim's algorithm to convert the graph into a minimum spanning tree. A few arcs are re-added to this minimum spanning tree to ensure that the maze generated is more difficult to solve.
11     #At the same time a tempGraph is produced from the new maze generated. This is so that I can use networkx's check_planarity function to ensure that the maze is planar. This is crucial for the displaying of the maze (to minimise overlapping arcs).
12     print(g)
13     self.maze = self.convertGraphToMaze(g)
14     tempGraph = convertAdjacencyMatrixToGraph(self.maze)
15     while nx.check_planarity(tempGraph)[0] == False:
16         self.maze = self.convertGraphToMaze(g)
17         tempGraph = convertAdjacencyMatrixToGraph(self.maze)
18     self.shortestPath = "NOT FOUND"
19     self.shortestPathLength = -1
```

The only additional thing I added was a planarity check. A graph is considered planar if it can be drawn with no overlapping arcs. Since no corridors in a maze overlap I believed it would be suitable to implement a planarity check. However, due to the extreme complexity of manually creating a planarity check and the numerous edge cases involved, I decided that this was not a feasible task - creating a planarity check is not the focus of my project. However, networkx, which is the library that I will be using for UI later on in the project, did have a built in planarity check which I use (lines 13-17.) These lines use my convertGraphToMaze function (which involves all the algorithms previously discussed) to convert the graph into a maze. This process repeats itself until the planarity check is passed. The line:

tempGraph = convertAdjacencyMatrixToGraph(self.maze)
Converts the adjacency matrix for the maze into a format that networkx recognises in order to actually carry out the planarity check. The final two lines define the attributes of the instance of the maze class for the shortest path. These attributes are populated with values when the class's dijkstras algorithm is performed:

```
1    def dijkstrasAlgorithm(self):
2        #This function runs the Dijkstras algorithm on the maze.
3        #The function first runs a forward pass where labels are produced for each node.
4        #The labels can immediately be used to determine
5        labels = self.performForwardPass(self.maze)
6        self.shortestPathLength = labels[len(self.maze)-1][1]
7        self.shortestPath = self.performBackwardsPass(labels, self.maze)
```

This algorithm is quite large so I decided to split it into two parts, one for the forward pass and one for the backward pass. The forward pass algorithm looks like this:

```
1    def performForwardPass(self, graph):
2        #Any usage of vNodes is used to display dijkstra's algorithm on the screen as the forward pass is being carried out.
3        vNodes = []
4        labels = {}
5        for index, rowNum in enumerate(graph):
6            #ORDER NUMBER, PERMANENT LABEL, TEMPORARY LABEL
7            labels[index] = [-1,-1,CONST_BIGNUMBER]
8        currentNode = 0
9        vNodes.append(str(currentNode))
10         #Node 0 (starting node) is given order number 1 and a permanent and temporary label of 0.
11         ordering = 1
12         labels[currentNode] = [ordering,0,0]
13         ordering += 1
14         #The function loops through each node and updates temporary labels of neighbors, then selects the next node with the smallest
temporary label (that isn't already fixed) to become a fixed node.
15         for i in range(len(graph)):
16             for rowIndex, row in enumerate(graph):
17                 for columnIndex, column in enumerate(row):
18                     if columnIndex == currentNode:
19                         if column != -1:
20                             newPotentialTempLabel = column + labels[currentNode][1]
21                             if (newPotentialTempLabel < labels[rowIndex][2]):
22                                 labels[rowIndex][2] = newPotentialTempLabel
23             labelIndexToBecomeNextNode = -1
24             newSmallestLength =CONST_BIGNUMBER
25             for labelIndex in labels:
26                 if labels[labelIndex][0] == -1:
27                     if (labels[labelIndex][2] < newSmallestLength):
28                         newSmallestLength = labels[labelIndex][2]
29                         labelIndexToBecomeNextNode = labelIndex
30             if labelIndexToBecomeNextNode != -1:
31                 labels[labelIndexToBecomeNextNode] = [ordering,labels[labelIndexToBecomeNextNode][2],
labels[labelIndexToBecomeNextNode][2]]
32                 currentNode = labelIndexToBecomeNextNode
33                 vNodes.append(str(currentNode))
34                 ordering += 1
35                 demonstrateDijkstras(vNodes)
36         return labels
```

To begin the forward pass we define 2 data structures, a list of visited nodes and a map for the labels. The vNodes list serves no purpose to the algorithm itself and is used for the visualization of the algorithm. The map of labels is in the format:

Key: The node number that is currently being visited
Value: A list of 3 items important to the label: ORDER NUMBER, PERMANENT LABEL, TEMPORARY LABEL

An integer representing the currentNode in the algorithm is set by default to 0, because the algorithm will start from node 0. Also ordering is used to track when a node has been turned into a fixed node during the running of the forward pass. Node 0 is added to the list of

vNodes and the label for node 0 is populated indicating that it has become the first fixed node. Now, a loop runs once for every node in the graph. This loop can be broken into two stages where in the first stage, a nested FOR loop is used to update the temporary labels of all the neighboring nodes to the currentNode. In the second stage, the algorithm loops through all the labels and finds the non-fixed node with the lowest temporary label. The algorithm now populates all the labels for this node and sets currentNode to this node to indicate that this node has been fixed so that in the next loop of the algorithm, the neighbours of this node are being updated. Finally, after the main FOR loop finishes, the labels are returned containing the necessary information to construct the shortest path and its length. Once the labels are returned to Dijkstra's algorithm, the length of the shortest path can immediately be determined from the permanent label of the largest node. To find the actual shortest path, a backwards pass needs to be performed on the labels:

```
1 def performBackwardsPass(self, labels, graph):
2       #This function now takes in the labels produced by the forward pass and uses them alongside the adjacency matrix to determine the shortest path.
3       reversedOrder = []
4       currentIndex = len(graph)-1
5       reversedOrder.append(str(currentIndex))
6       nodeFound = False
7       #The function loops through the adjacency matrix. It then calculates the difference between the permanent label of the current node and the permanent label of all neighbours (to determine the previous node in the path).
8       #If this difference is equal to the length of the arc between these nodes (found using the matrix), then the previous node is now taken as the current node. The previous node is now added to the reversedOrder list.
9       while currentIndex != 0:
10          for rowIndex, row in enumerate(graph):
11              for columnIndex, column in enumerate(row):
12                  if (nodeFound == False):
13                      if columnIndex == currentIndex:
14                          if (graph[rowIndex][columnIndex] != -1):
15                              difference = labels[currentIndex][1] - labels[rowIndex][1] - graph[rowIndex][columnIndex]
16                              if difference == 0:
17                                  currentIndex = rowIndex
18                                  reversedOrder.append(str(currentIndex))
19                                  nodeFound = True
20          nodeFound = False
21      #At this point the reversedOrder list contains the order of the nodes in the shortest path in reverse.
22      #This for loop reverses the order of the nodes in the reversedOrder to find the actual shortest path.
23      correctOrder = ""
24      for index,i in enumerate(reversedOrder):
25          if  index == 0:
26              correctOrder = str(reversedOrder[len(reversedOrder)-1-index])
27          else:
28              correctOrder = correctOrder + "," +  str(reversedOrder[len(reversedOrder)-1-index])
29      return correctOrder
```

This algorithm begins by defining a reversedOrder list and appending the largest node in the list to this reversed order. This is because the backward pass always begins at the final node. Now, a WHILE loop occurs that loops constantly until the algorithm has made it to the first node. Inside the loop, the algorithm checks all connections from the current node to its neighbouring nodes. If the permanent label of the current node is equal to the permanent label of one of its neighbouring nodes added to the distance between the two nodes, then that neighbouring node is the next node in the reversed path. By following and repeating this rule, the reversed shortest path can be deduced. After the while loop is finished, the reversedShortest path needs to be reversed to give the shortest path. This is done in the final FOR loop. To format the final path in a nicer manner, the final shortest path is provided as a string of node numbers separated by commas. When coding this algorithm, I coded it using primarily my knowledge of further maths which involved both the forward and backward pass. This version of the algorithm is inefficient and has a time complexity of

O(n^2) + O(n^2) + O(n) - one for the forward pass, one for the backward pass and one for the reversing of the final string. Had I optimized the algorithm to store the previous fixed node for each temporary node this algorithm would only have a complexity of O(n^2) - for just the 1 pass. This would have made my program cleaner, more efficient and arguably easier to program. For demonstrational purposes, the original version of Dijkstra's algorithm remains in the program, however this serves as a critical learning point to spend more time designing and researching into an algorithm before going all in and coding it. Here is how the optimised version of the entire Dijkstra's algorithm would look:

```
1   def optimalDijkstras(self, graph):
2       #Any usage of vNodes is used to display dijkstra's algorithm on the screen as the forward pass is being carried out.
3       vNodes = []
4       labels = {}
5       for index, rowNum in enumerate(graph):
6           #ORDER NUMBER, PERMANENT LABEL, TEMPORARY LABEL, PREVIOUS NODE
7           labels[index] = [-1,-1,CONST_BIGNUMBER, -1]
8       currentNode = 0
9       vNodes.append(str(currentNode))
10      #Node 0 (starting node) is given order number 1 and a permanent and temporary label of 0.
11      ordering = 1
12      labels[currentNode] = [ordering,0,0, -1]
13      ordering += 1
14      #The function loops through each node and updates temporary labels of neighbors, then selects the next node with the smallest
temporary label (that isn't already fixed) to become a fixed node.
15      for i in range(len(graph)):
16          for rowIndex, row in enumerate(graph):
17              for columnIndex, column in enumerate(row):
18                  if columnIndex == currentNode:
19                      if column != -1:
20                          newPotentialTempLabel = column + labels[currentNode][1]
21                          if (newPotentialTempLabel < labels[rowIndex][2]):
22                              labels[rowIndex][2] = newPotentialTempLabel
23                              labels[rowIndex][3] = currentNode
24          labelIndexToBecomeNextNode = -1
25          newSmallestLength =CONST_BIGNUMBER
26          for labelIndex in labels:
27              if labels[labelIndex][0] == -1:
28                  if (labels[labelIndex][2] < newSmallestLength):
29                      newSmallestLength = labels[labelIndex][2]
30                      labelIndexToBecomeNextNode = labelIndex
31          if labelIndexToBecomeNextNode != -1:
32              labels[labelIndexToBecomeNextNode] = [ordering,labels[labelIndexToBecomeNextNode][2],
labels[labelIndexToBecomeNextNode][2], labels[labelIndexToBecomeNextNode][3]]
33              currentNode = labelIndexToBecomeNextNode
34              vNodes.append(str(currentNode))
35              ordering += 1
36              #demonstrateDijkstras(vNodes)
37      print(labels)
38      reversedOrder = []
39      currentIndex = len(labels)-1
40      while currentIndex != 0:
41          print(currentIndex)
42          reversedOrder.append(str(currentIndex))
43          currentIndex = labels[currentIndex][3]
44      reversedOrder.append("0")
45      shortestPath = []
46      for index, i in enumerate(reversedOrder):
47          shortestPath.append(reversedOrder[len(reversedOrder)-1-index])
48      print("NewAlgorithmShortestPath:")
49      print(shortestPath)
```

At this point in the program I have completed all the necessary setup steps as stated in the brief 3 stage plan. I am now able to begin creating an AI to solve the maze.

AI maze solving capabilities

To begin I started by created a new AI Maze Solver class and defined important attributes to the functionality of the AI:

```
1 class AIMazeSolver:
2    #Generally I would advise to avoid modifying the constants as it leads to unexpected results. For instance, if the learning rate is too high (more than 0.5) the Q function will not converge to a solution.
3    #Adjustable constants
4    CONST_EPSILONDECAYRATE = 0.995
5    CONST_LEARNINGRATE = 0.1
6    CONST_DISCOUNTFACTOR = 0.9
7    #End of adjustable constants
8    def __init__(self, mazeToSolve):
9        #This function is called when an instance of the class Maze is created. It will create new public variables. These are the tabular Q function, the path found by the Q function, and the adjacency matrix of the maze to solve.
10       self.tabularQFunction = []
11       self.pathFound = ""
12       self.mazeToSolve = mazeToSolve
```

The 3 constants I defined so that it would make the training process more readable (rather than hardcoding numeric constants throughout the code.) This also makes it easier to modify if any other programmer wanted to experiment with the AI, however, after testing, I have found that these particular values work best to find the shortest path in a reasonable time. I have also defined the actual tabularQFunction which, as explained in my analysis, will act as the ‘brain’ of the AI. The format of tabularQFunction (2D list) is:

```
[
        [currentNode, potentialNextNode, QFunctionValue]
]
```

Pathfound, intuitively, is the path found by the AI and mazeToSolve is the maze being inputted into the solver.

Before going into the logically heavy process of training and testing an AI, I decided to quickly implement several quality of life algorithms that would greatly improve the ease of coding as well as the efficiency of the program. One of these functions is getNeighbours:

```
1    def getNeighbours(self, graph, index):
2        #This function is used to find the neighbours of a node in the graph using the matrix.
3        neighbours = []
4        for rowIndex, row in enumerate(graph):
5            for columnIndex, column in enumerate(row):
6                if (columnIndex == index):
7                    if (column != -1):
8                        neighbours.append(rowIndex)
9        return neighbours
```

Simply put, this subroutine returns a list of all neighbours of a particular node in the maze. It does this by looping through every entry in the adjacency matrix and checking whether or not this entry is in the selected column (belonging to the node) and is not equal to -1 (indicating no edge.) If both these conditions are met, the row index of this entry is appended to the list. Amidst coding my trainQFunction subroutine, I also created another quality of life algorithm, getNodeWithHighestQFunctionValue:

```
1    def getNodeWithHighestQFunctionValue(self, tQF, node):
```

```
2       #This function passes through the tabular Q function and returns the state action pair for the specified node with the highest Q function
value.
3       greatestQV = "Not found"
4       potNextNode = "Not found"
5       for row in tQF:
6           if row[0] == node:
7               if greatestQV == "Not found":
8                   potNextNode = row[1]
9                   greatestQV = row[2]
10              else:
11                  if row[2] > greatestQV:
12                      potNextNode = row[1]
13                      greatestQV = row[2]
14      return [greatestQV, potNextNode]
```

This program loops through every row in my table. This function loops through every row in the Table and finds the row corresponding to the input node that has the largest Q-Function value. The function then returns a list containing this Q-Function Value and the neighbouring node that would give this Q-Function value.
With quality of life algorithms complete I can now begin training my AI using the trainQfunction subroutine:

```
1  def trainQFunction(self):
2       #This function is used to train the tabular Q function for the AI.
3       #Epsilon is responsible for slowing causing the AI to shift from exploration of new nodes to the exploitation of previous know paths.
4       epsilon = 1
5       epsilonMinimum = 0.05
6       #At the beginning of the function the tabular Q function is populated with the initial values.
7       for rowIndex, row in enumerate(self.mazeToSolve):
8           for columnIndex,column in enumerate(row):
9               if column != -1:
10                  # [currentNode, nextNode, Q-value]
11                  qFunctionInsert = [rowIndex, columnIndex, 0]
12                  self.tabularQFunction.append(qFunctionInsert)
13      #Q(s,a) = Q(s,a) + alpha*(r + gamma*max(Q(s',a')) - Q(s,a))
14      count = 1
15      self.pathFound = ""
16      keepLooping = True
17      pathHistory = []
18      #Each loop in this while loop represents an episode in the training of the agent.
19      while keepLooping == True:
20          self.pathFound = ""
21          count += 1
22          currentNode = 0
23          self.pathFound = str(currentNode)
24          #In each episode, the AI starts at the start node. This while loop ensures that the AI keeps moving until it reaches the end node.
25          while (currentNode != len(self.mazeToSolve)-1):
26              nextNode = -1
27              #In each episode, the AI will either perform an explore or an exploit step. In an explore step, the agent will make a completely
random decision as to where it will move. In an exploit step, the agent will move to the node with the highest maximum Q function value.
28              if (random.random() < epsilon):
29                  #Explore step
30                  neighbours = self.getNeighbours(self.mazeToSolve, currentNode)
31                  nextNode = neighbours[random.randint(0,len(neighbours)-1)]
32              else:
33                  #Exploit step
34                  nextNode = self.getNodeWithHighestQFunctionValue(self.tabularQFunction, currentNode)[1]
35              #After each step, the Q function is updated so that the Q value for the current state action pair better suits the bellman equation.
36              for row in self.tabularQFunction:
37                  if row[0] == currentNode and row[1] == nextNode:
38                      row[2] = row[2] + self.CONST_LEARNINGRATE*(-self.mazeToSolve[currentNode][nextNode] +
self.CONST_DISCOUNTFACTOR*self.getNodeWithHighestQFunctionValue(self.tabularQFunction, nextNode)[0] - row[2])
39              currentNode = nextNode
40              self.pathFound = self.pathFound + "," + str(currentNode)
41          #After each episode, the path found by the AI is displayed.
42          if (count % 1 ==0):
43              showPath(self.pathFound, False)
44          #The AI keeps running more training episodes until the training yields the same path for 20 episodes in a row.
45          if len(pathHistory) < 20:
46              pathHistory.append(self.pathFound)
```

```
47          else:
48              pathHistory.pop(0)
49              pathHistory.append(self.pathFound)
50              consistent = True
51              firstPath = pathHistory[0]
52              for path in pathHistory:
53                  if path != firstPath:
54                      consistent = False
55              if consistent == True:
56                  keepLooping = False
57          epsilon = max(epsilonMinimum,epsilon*self.CONST_EPSILONDECAYRATE)
```

To set up the AI for training, a couple things need to be done. Firstly, I create a variable epsilon, which will be initially set to 1 so that the AI is guaranteed to start exploring the maze. I also set a minimum value of epsilon (0.05) to ensure that even in later stages of the training, occasionally the AI still makes an exploration step. I used nested FOR loops to set up the tabular-q-function so that it has all possible state-action pairs and a corresponding q-value of 0. The variable count is used for visualisation and allows me to adjust how frequently a training episode is being displayed. Our previously defined pathFound variable is set to an empty string, ready to be updated when the path is found. keepLooping and pathHistory are used to stop the training when the algorithm has converged to a shortest path. The way they are used is, if the last 20 training episodes have yielded the same path, then the AI has most likely converged on the optimal shortest path and so keepLooping is set to false. The training episodes begin inside the main WHILE loop (while keepLooping == True:). In each episode the pathFound is set to a string containing the first node (0). This first node is set to currentNode. An inner while loop is now used to ensure that the episode ends on the final node. This inner while loop carries out each step in one episode of training. In each step, a random number between 0 and 1 is generated and is compared to the value of epsilon. If the value is less than epsilon, then an exploration step will be taken. In this step, the agent will select a random neighbouring node and traverse to it, indicating a random step. If the random number is greater than epsilon, then an exploitation step will be taken. In this step, the neighbouring node with the largest Q-Function is selected and traversed to. After the step has been completed, the current node and the next node are used in the Bellman equation to update the q-value of the state-action pair. One thing to note in the Bellman equation is the distance is given as a negative value. This negative value is because we want to punish the agent for selecting longer paths, which is significantly easier than rewarding the agent for selecting shorter paths and provides the same effect. This happens regardless of whether the step taken was an exploration step or an exploitation step. At the end of the step,the path used is updated to include the current node and the next node becomes the current node. At the end of each episode, the path traversed is passed into the subroutine showPath in order for the visualisation of training to work. The path selected in the episode is added to the pathHistory list and then a FOR loop is used to check whether the last 20 training episodes produced the same path (training stops.) Finally, epsilon is updated using the epsilon decay rate to incentivise exploitation over exploration in later episodes of the training. The training of the AI is now complete and the table is now capable of solving this maze almost instantly. To see how effective the agent is at solving the maze I have created an additional testQfunction subroutine:

```
1  def testQFunction(self, newMaze):
2      #This function is used to provide the AI with data for a new potential maze. The AI now uses the tabular Q function to find the shortest path to the end node. It does this by only performing exploit steps.
3      #The Q-Values are not updated in this function.
4      cNode = 0
5      nNode = -1
```

```
6       pFound = str(cNode)
7       while (cNode != len(newMaze)-1):
8           nNode = -1
9           nNode = self.getNodeWithHighestQFunctionValue(self.tabularQFunction, cNode)[1]
10            cNode = nNode
11            pFound = pFound + "," + str(cNode)
12        showPath(pFound, True)
13        return pFound
```

This function acts in a very similar way to 1 training episode written above; however, the agent will now exploit its tabular-q-function 100% of the time and hence will no longer take any random steps in any direction. Furthermore, the q-values in the table are no longer updated, this ensures that the AI's performance is evaluated purely on knowledge gained during training. By always selecting the neighbouring node with the highest Q-Function, the AI consistently finds the shortest path.

From a theoretical perspective, the program is complete, however I deemed it necessary from an educational point of view (and to follow my objectives) to create a visualisation of a few of the core algorithms in this program.

User interface

Here are the libraries that I imported into my computer program.

```
1 import copy
2 import random
3
4 import matplotlib.pyplot as plt
5 import networkx as nx
```

The library copy and random were both used during maze generation and serve no purpose for the user interface. Matplotlib is a general purpose plotting library which is generally used to visualise data (e.g. 2D charts.) Networkx allows me to work with graphs found in discrete mathematics (the graphs that we are using in this project.) In order to make the code responsible for visualisation more concise, I created a function that would allow me to easily convert any adjacency matrix into a graph recognised by networkx:

```
1 def convertAdjacencyMatrixToGraph(graph):
2     #This function converts an adjacency matrix into the graph data type that is used by networkx. The function determines the weight of the arc between any two nodes and if it is not -1, it adds it to the graph.
3     G = nx.Graph()
4     for rowIndex, row in enumerate(graph):
5         for columnIndex, column in enumerate(row):
6             weight = graph[rowIndex][columnIndex]
7             if weight != -1 and weight != CONST_BIGNUMBER:
8                 G.add_edge(columnIndex, rowIndex, weight=weight)
9     return G
```

This code loops through every entry in the adjacency matrix and if the entry is not -1, the algorithm adds a new arc to the Networkx custom graph data type corresponding to the connection between these two nodes.

During the running of Dijkstra's algorithm, the function demonstrateDijkstras was called every time a new fixed node was created. Here is the function:

```
1 def demonstrateDijkstras(stringVisitedNodes):
```

```
    #This function is used to demonstrate the forward pass of the dijkstra's algorithm. At each stage of the algorithm, the previous fixed nodes are highlighted in yellow.
    #This for loop identifies all the nodes visited by the algorithm.
    visitedNodes = []
    for stringNode in stringVisitedNodes:
        visitedNodes.append(int(stringNode))
    #This loop stores the colour of each node depending on whether or not the node has been visited by the algorithm. Yellow is for a visited fixed node and lightblue is for a non fixed node.
    nodeColours = []
    for node in G.nodes():
        if node in visitedNodes:
            nodeColours.append("yellow")
        else:
            nodeColours.append("lightblue")
    #These are functions that are part of matplotlib.pyplot and networkx libraries used to display the graph. First the previous graph is cleared, and the new graph is then displayed with the updated visited nodes. The weight of each arc are added to this new graph.
    #I added a slight delay of 0.5 seconds so that the user can see the changes to the graph as the algorithm progresses.
    plt.clf()
    nx.draw(
        G, pos, with_labels=True,
        node_color=nodeColours,
        node_size=200
    )
    labels = nx.get_edge_attributes(G, 'weight')
    nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
    plt.pause(0.5)
```

This subroutine converts the string of visited nodes into a list of visited nodes (as integers). Then for every node in the graph, if the node has been visited, its colour is set to yellow, otherwise it remains blue. plt.clf() clears the previous graph and nx.draw() draws the new graph. nx.draw_networkx_edge_labels adds the weights of the labels to this new drawn graph. Because the computer performs the algorithm so quickly, I added a pause function into the visualisation so that people can actually see the algorithm being performed.

At the end of Dijkstra's algorithm and throughout the trainQFunction subroutines you could see a showPath procedure being run:

```
def showPath(path, isFinalPath):
    #This function is used to display the shortest path found by the dijkstra's algorithm and the paths taken by the AI to find this optimal path. This function is similar to the demonstrateDijkstras function however additionally displays the arcs that are part of the shortest path.
    #Any arcs and nodes that are passed through during the training of the AI will be highlighted in green. Any arcs and nodes that are part of dijkstra's final path or that are part of the AI's final path found during testing will be highlighted in red.
    stringVisitedNodes = path.split(",")
    #This for loop identifies all the nodes visited by the algorithm.
    visitedNodes = []
    for stringNode in stringVisitedNodes:
        visitedNodes.append(int(stringNode))
    nodeColours = []
    for node in G.nodes():
        if node in visitedNodes:
            if (isFinalPath == False):
                nodeColours.append("green")
            else:
                nodeColours.append("red")
        else:
            nodeColours.append("lightblue")
    #This for loop identifies all the edges visited by the algorithm.
    edgesInPath = []
    for i in range(len(visitedNodes) - 1):
        edgesInPath.append((visitedNodes[i], visitedNodes[i + 1]))
    edgeColours = []
    for edge in G.edges():
        if edge in edgesInPath or (edge[1], edge[0]) in edgesInPath:
            if (isFinalPath == False):
                edgeColours.append("green")
            else:
                edgeColours.append("red")
        else:
            edgeColours.append("lightgray")
```

```
31    #As described previously, these functions are part of matplotlib.pyplot and networkx libraries used to display the graph.
32    plt.clf()
33    nx.draw(
34        G, pos, with_labels=True,
35        node_color=nodeColours,
36        edge_color=edgeColours,
37        node_size=200
38    )
39    labels = nx.get_edge_attributes(G, 'weight')
40    nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
41    if (isFinalPath == False):
42        plt.pause(0.01)
43    else:
44        plt.pause(1)
45        input()
```

This procedure, although significantly larger, works similarly to demonstrateDijkstras. A path is inputted alongside whether or not it actually is the final path selected by the algorithm. If the path is final the nodes and arcs corresponding to this path will be displayed in red. Otherwise they will be displayed in green. As in the previous algorithm, showPath first converts the string containing the path into a list of integer nodes representing the path. In the first FOR loop, the algorithm identifies the nodes in the shortest path and provides them with the necessary colour, and leaves the remaining nodes as blue. The code performs a similar operation in the second FOR loop however, this time on all arcs connecting nodes in the maze. Then, the same form of visualisation is carried out as in demonstrateDijkstras. However, this time, the time taken to pause varies. If isFinalPath is false, the time delay between displays is only 0.01s. This is because isFinalPath is false during the training of the agent, and since there are so many training episodes, they need to be performed in quick succession. If isFinalPath is true, there is a time delay of 1 second followed by an input. This input is to give the user time to analyse the finalPath before continuing - whether this is to continue to the training stage, or to close the program.

Finally, this code is strewn together in the main part of my program:

```
1 #This creates a new instance of the Maze class - new maze is randomly generated.
2 MazeCl = Maze()
3 maze = MazeCl.maze
4 #This maze is converted into a networkx graph to be displayed.
5 G = convertAdjacencyMatrixToGraph(maze)
6 #This positions the start and end nodes of the maze so that the start node is on the left hand side of the screen and the end node is on the right
hand side.
7 fixed_positions = {
8         0: (-1.5, 0),
9         len(maze) - 1: (1.5, 0)
10    }
11 #This is the layout of the graph I decided to use.
12 pos = nx.spring_layout(G, pos=fixed_positions, fixed=fixed_positions.keys())
13 #This is the figure size of the graph.
14 plt.figure(figsize=(15, 9))
15 #The maze is now solved using dijkstras algorithm. Once dijkstras algorithm is complete, the shortest path is displayed.
16 MazeCl.dijkstrasAlgorithm()
17 showPath(MazeCl.shortestPath, True)
18 #The maze is now solved using AI.
19 AI = AIMazeSolver(maze)
20 AI.trainQFunction()
21 AI.testQFunction(maze)
```

Firstly, a new random maze is generated and converted into a graph accepted by NetworkX. Fixed_positions fixes the location of the start and end nodes on the screen so that they are easier to find for users. Line 12 is a NetworkX layout that arranges my graph's nodes in a visually appealing manner. Line 14 tells Matplotlib to actually create a new display in a new

window. Dijkstra's algorithm is then carried out on the new maze and the shortest path found is shown. The AI is then created and trained on this maze. It is then tested on the maze, and the path found is displayed on the screen.

Conclusion

I set out in this project to determine whether or not reinforcement learning could be used to create an effective AI Maze-solver. I successfully implemented algorithms responsible for mathematically generating and solving a maze. I have also successfully created a RL model in Python that was capable of navigating a maze to find the optimal path. My model found the optimal path consistently, occasionally finding a different path to dijkstra's if there were 2 identical shortest paths. Upon experimentation primarily through trial and error, I have found the optimal AI hyperparameters that would cause the agent to converge on the correct solution in the shortest time possible. I have largely achieved my main goal of helping my brother to better understand maze solving. This project has sufficiently explored the foundational concepts that underpin modern AI systems and is extremely relevant in the context of the current surge in artificial intelligence. Due to large time constraints, I was not able to create a Deep Q-Network which would have allowed the agent to deal with larger state spaces more efficiently. However, given that I created a Tabular Q-Function in vanilla Python (no libraries for AI), I believe this was more than enough to meet the aims of my project.

Evaluation

I am now nearing the end of my EPQ and I think it is a good time to reflect. I believe the project has largely been successful, and I now have an AI that is capable of solving a randomly generated maze. Entering this project, here were my main objectives:

1. Develop a maze generation system that can produce random mazes of varying sizes and complexities using a combination of algorithms present in discrete mathematics.
This objective I believe I completed successfully. In my research review I gave an in depth breakdown of how each of my maze generation algorithms work. I then actually implemented the algorithms inside my final project and explained how I implemented them.

2. Implement Dijkstra's algorithm to solve the maze deterministically to provide a benchmark to compare that AI against and evaluate its performance.
This was another objective that I completed successfully. I explained both how it worked and its implementation. Initially I was planning on also implementing the A* algorithm, a heuristic algorithm based on Dijkstra's, in my project. However, I ruled this out due to the unnecessary additional complexity this would bring to my project - mazes are too small for it to make a noticeable difference in the time taken for the computation and the algorithm is too complex.

3. Create and train an AI model using reinforcement learning (q-function/deep q-function) that learns to navigate the maze using trial and error.

This objective was one of my largest fears entering this project as I had completely no prior experience regarding these AI models. Entering this project I wanted to make a Deep Q-Network as I believed it would stretch my knowledge and maximise the technical capabilities of the AI. This ambitious goal drove extensive research into these deep learning concepts. After seeing the sheer scale the project would need to take in order to implement this AI model, I made a well-thought out modification to create a tabular q-function even if this did interfere with my last objective. I did in depth research into this type of AI and ended up successfully implementing this in my code (the agent would consistently find the same shortest path that Dijkstra's algorithm would find).

4. Observe any maze solving techniques some of the later generations of AI have used to solve the maze and compare them with maze theory used nowadays (game theory).

This objective, I ultimately judged as unachievable and this was a consequence of strategic scope adjustment during the research stage of this project. This objective was deliberately ambitious, and was created in this way to push me into high-level research into Deep-Q-Networks. A DQN was intended to allow the agent to understand the general structure of mazes thus generalising maze solving. A DQN would learn transferable policies such as prioritising visiting nodes with a high quantity of neighbours. I would then observe these policies and begin to translate them into maze-solving tips and tricks that would be applicable in real life. However, after doing some research into how DQN worked, I understood that the objective of creating a DQN was not going to be achievable in the 6 month timeframe. Hence I made a calculated modification and instead made the Tabular Q-Function.

Research

My research phase was arguably the most important part of my project. I already came into the EPQ with a strong ability to program so any algorithm that I was to make was not going to be too difficult to write. However, before I could write anything I had to get to grips with the concepts behind what I was writing. I also had to simultaneously begin carrying out more thorough planning into how I was going to create this project. To some degree, explaining the maze generation process was quite simple due to my prior knowledge of further maths and computer science. Even so, I had to make a couple important choices between algorithms. For example, when verifying that the random adjacency matrix was connected, I had to make a choice between a breadth-first-search or a depth-first-search. Another example of this was when I had to choose between using Kruskal's algorithm or Prim's algorithm when converting the connected graph into a minimum spanning tree. These decisions, I had to make on a variety of factors such as time complexity; space complexity and ease of implementation. Initially I was tempted to do the A* algorithm as it is by far the fastest way to find a short path through a graph but due to this not being the primary focus of my project (and being quite difficult to make), I ruled this out. This is one of the recurring issues that I had with this project and that was aiming unrealistically high. As mentioned previously this initial over aiming was good for my project as it drove high-level technical research however, it meant that I had to strategically reform some of my main objectives to better suit my feasible technical ability. Therefore, I slightly held myself back during the algorithmic maze solving and stuck to a more reasonable Dijkstra's algorithm. Honestly, sometimes writing up and explaining how these algorithms work was quite a tedious and boring process. I personally am much more of a math and programming oriented person (as

you can gauge by my A-Level choices) so I often lost focus during this phase. At this stage I also began learning about how reinforcement learning works and this was extremely difficult for me as this was one of the topics I had no prior knowledge in. After about a week of watching educational tutorials and reading about the topic on certain websites I was a lot more comfortable with the ideas and was able to write this information into my research review.

Development Process

After doing research on what to do, I actually had to go ahead and program the project. I planned to do this over the summer holiday as I thought this would give me the most amount of time to do this. I planned to carry out my programming in rigorous daily chunks of about 6-7h. This may seem counterproductive but I was advised by my father (who himself is a software developer) that this was the most effective way to do this because context switches can be detrimental to my progress. I also program as a hobby so these long hours went by relatively quickly in the heat of coding. I found coding the algorithms to set up the maze fairly easy and this gave me confidence that I would be able to complete the programming side of the project in the time frame that I had initially planned out. There were a few minor bugs here and there but this is to be expected with most programming projects and I didn't believe these bugs to be important enough to be included in the detailed documentation. I found coding the AI a difficult task and this was something that I expected to happen but my planning gave me more time to work through this side of my project. I did make a few major mistakes during this process and these I documented in my activity log. Although difficult, this was by far the most enjoyable part of the project and the essence of why I selected this EPQ artefact. I wanted to stretch my programming skills and understand how I would go about making an AI and I was actively doing this at this stage. I was also very happy with the result of this AI as it exceeded my initial expectations of what I was going to do. Going into the project I was more than certain that I was going to simply import some python library that had the Tabular Q-Function well defined and that I would simply have to tweak some input parameters in order to get this AI working for my project. I on the other hand coded the entire AI in pure python without using anyone else's code which for me was a huge achievement. Even with this being a difficult task, I still managed to get it done in less time than I had initially planned which was fortunate. This gave me more leeway to experiment with different libraries for visualisation. I searched through a variety of libraries and public documentation as well as discussed my needs with LLMs such as ChatGBT and Gemini and eventually landed on MatplotLib and NetworkX, which I have provided in my Research sources. This was slightly different to my initial plan to use Pygame but I think this change and moreover my flexibility to changes saved me a lot of time in this project. These libraries I found had extensive documentation and after a bit of experimentation, I found the ideal visualization settings for my project. This was the only task that required me to be largely dependent on the internet. This is not necessarily bad because using external sources is necessary in programming as is it necessary in an EPQ but I had to be careful not to directly copy and paste large amounts of code directly in the project. Finally for completeness, I went through my project and just added comments. Comments are often a largely ignored part of programming because they don't actually add anything to the final output, but they are largely useful for other people trying to read and understand the code. I didn't like doing this final bit but I considered it necessary for completeness.

Difficulties and how I overcame them

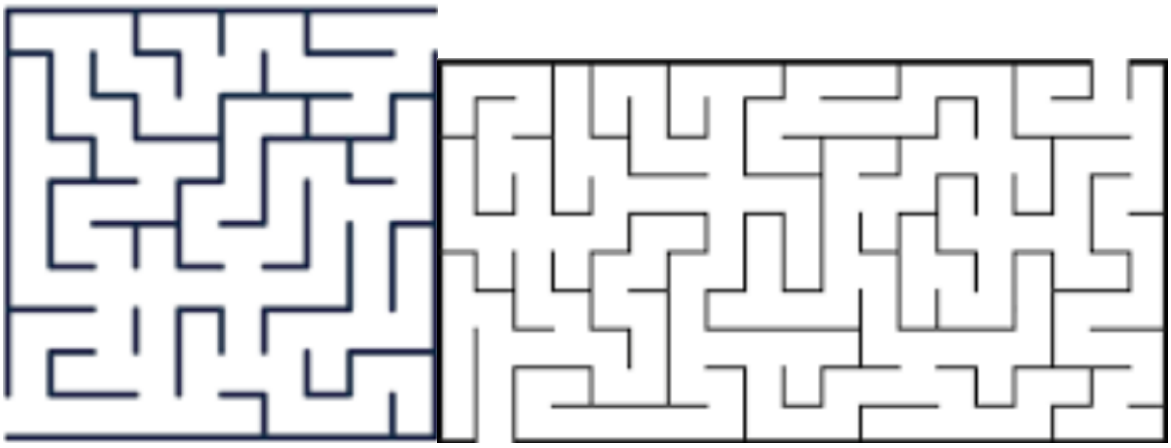
One of the largest difficulties for me was juggling a large amount of external commitments. Amidst working on this EPQ, I had to simultaneously do my computer science NEA which was also a large programming project. I also had to spend a lot of time revising for the TMUA as I wanted to get a high score in this to have a competitive chance for some of the universities I was applying to. Once I had done this I had to prepare for my university interviews. Alongside this, I was continuously balancing revision for 4 academically rigorous A-Levels. Over this period I had a lot of things to deal with all at once and this meant I was often tired and stressed. I think I did two things that largely helped me out over this period. Firstly, a lot of the programming for both of my projects I did over the summer holiday. This largely reduced my workload during the college hours - which allowed me to concentrate my revision on A-Levels and exam preparation. Another thing that I did was plan my time well. This may seem somewhat self-explanatory but having a plan enabled me to focus 100% of my time on one particular thing. For example, 2 weeks before the TMUA I had planned to do no EPQ and no computing coursework because I wanted to focus 100% of my time on just revision for the exam. I have found that multitasking is not one of my strong points and really was just a source of stress so in doing this, I helped myself to calm down.

Another unexpected issue was actually getting the code onto the school PC's. In my plan I considered this to be a minor hiccup in my EPQ but in reality this was a very large issue that I tried to fix, spanning over a couple days. The school PC's had such extensive security features that it made it difficult to do basically anything - barring what the PC's were designed to do. Even with my father's guidance I was still unable to get the code working on the school PC's. I put in more detail on what I did to try and overcome these issues on my activity log. I ended up overcoming this issue by using an online IDE (google collab) to run the code on external devices. For a nicer visualisation during my presentation, I actually brought in my laptop and ran the code on the laptop to demonstrate my algorithms and training process working. Ironically, these small, unexpected, time-killing issues are to be expected with every programming project. In my computer science NEA I thought I was finished with my coding only to spend hours compressing a multitude of educational videos to meet some very small storage requirements.

These issues I had a vague idea of, entering this project, but the scale of each issue I had no idea about. Even still, I believe I handled both of these issues adequately and tried to avoid losing as much time as possible as time was a major finite resource over these 6 months.

What could have been improved and what would I have done differently

If I were to do this project again I might look at increasing the customisability of the program. Instead of having one fixed set of graph generation algorithms I might enable the user to choose between different algorithms that perform the same task but in a different manner. For example, instead of forcing the program to do breadth-first search, the user would choose between breadth-first search and depth-first search, or during the generation of the minimum spanning tree, the user would choose between Kruksal's algorithm and Prims. Due to time constraints I also had to restrict the visualization to just Dijkstra's algorithm and AI training and testing process. With a few weeks more time I definitely would have produced demonstrations of the graph generation process. With a lot more time (additional 2-3 months) I would have looked into a completely different maze visualization. Classical mazes look like this:



Whereas the mazes I have generated are effectively just coloured networks. Logically and algorithmically, nothing would be different between this maze representation and my maze representation but, this version of a maze is much more widely accepted in pop culture. The reason for not creating this kind of maze was that there were no readily available python libraries that would make a maze like this. Therefore, I would have to completely overhaul my current UI and create a new UI from scratch that would generate these types of mazes. Since making a nice UI was not the focus of this project I decided to not create this, but with more time, this definitely would have been a possibility I would have looked into. Saying this, coloured networks are not completely tragic. A traditional wall maze, while aesthetically pleasing, effectively hides the core graph theory responsible for the AI so this UI has its upsides and its downsides. This would take away from the educational factor which is present in the program currently. With double the time I received to do this EPQ I most likely would have created a Deep-Q-Neural network. This would have required significantly more research, explanation and understanding in order to create but the result would be an AI with much higher functional capabilities.